

# Kiến trúc SOA & Microservices

---

Đặng Ngọc Hùng



# Mục lục

---

|                                                                    |           |
|--------------------------------------------------------------------|-----------|
| <b>6. Giao dịch Phân tán — Saga Pattern . . . . .</b>              | <b>4</b>  |
| <b>6.1. Độ phức tạp của Giao dịch phân tán</b>                     | <b>5</b>  |
| <b>6.2. Saga Pattern — Chuỗi Local Transactions + Compensation</b> | <b>7</b>  |
| 6.2.1. Định nghĩa . . . . .                                        | 8         |
| 6.2.2. KBM Submit Saga . . . . .                                   | 8         |
| 6.2.3. Ba loại transactions trong saga . . . . .                   | 9         |
| <b>6.3. Choreography vs Orchestration</b>                          | <b>9</b>  |
| 6.3.1. Choreography — Phi tập trung, event-driven . . . . .        | 10        |
| 6.3.2. Orchestration — Tập trung, commanding . . . . .             | 10        |
| 6.3.3. Khi nào dùng gì? . . . . .                                  | 11        |
| <b>6.4. Compensating Transactions &amp; Isolation</b>              | <b>12</b> |
| 6.4.1. Thiết kế compensation trong KBM . . . . .                   | 12        |
| 6.4.2. Vấn đề isolation — Anomalies . . . . .                      | 13        |
| 6.4.3. Countermeasures . . . . .                                   | 15        |
| 6.4.4. Tổng hợp: Anomaly → Countermeasure . . . . .                | 16        |
| <b>6.5. Eventual Consistency — Chấp nhận và quản lý</b>            | <b>17</b> |
| 6.5.1. Consistency window trong KBM . . . . .                      | 17        |
| 6.5.2. Strategies cho UI và business . . . . .                     | 18        |
| <b>6.6. Case Study: KBM</b>                                        | <b>18</b> |
| 6.6.1. Hiện trạng: Implicit Saga (Choreography) . . . . .          | 18        |
| 6.6.2. Workflow học vụ cũng là saga . . . . .                      | 22        |
| <b>6.7. Tổng kết</b>                                               | <b>23</b> |
| <b>6.8. Đọc thêm</b>                                               | <b>23</b> |
| <b>Tài liệu tham khảo</b>                                          | <b>23</b> |



# 6.

## CHƯƠNG 6.

# Giao dịch Phân tán — Saga Pattern

---

“A saga is a sequence of local transactions. Each local transaction updates the database and publishes a message or event to trigger the next local transaction in the saga.”

— Chris Richardson, *Microservices Patterns* [2a]

### MỤC TIÊU HỌC TẬP

- Hiểu tại sao distributed transactions (2PC) không phù hợp với microservices
- Nắm vững Saga pattern: định nghĩa, cấu trúc, và compensating transactions
- So sánh hai coordination mechanisms: Choreography vs Orchestration
- Áp dụng countermeasures để xử lý thiếu isolation giữa sagas
- Hiểu eventual consistency và cách quản lý từ góc nhìn business
- Phân tích luồng nộp bài trong KBM — implicit saga và migration path

Việc chuyển đổi từ kiến trúc monolith nguyên khối sang môi trường phân tán của microservices mang lại nhiều lợi ích to lớn về khả năng mở rộng độc lập và triển khai linh hoạt. Tuy nhiên, kiến trúc này cũng đồng thời phá vỡ tính nhất quán dữ liệu truyền thống. Chương này sẽ khám phá những thách thức cốt lõi của giao dịch phân tán và cách sử dụng Saga pattern để đảm bảo hệ thống luôn vận hành chính xác và đồng bộ.

### Ôn lại Kiến thức Nền

Chương này dựa nhiều vào một số khái niệm cơ sở dữ liệu. Nếu chưa chắc, hãy nắm nhanh bốn ý sau (ví dụ trừ tiền trong thẻ sinh viên (ví e-learning) để nạp quota in ấn ở thư viện):

**Transaction** – một nhóm thao tác “tất cả hoặc không có gì”. Nạp quota phải *trừ* tiền thẻ sinh viên và *cộng* quota in ấn cùng lúc; không thể trừ tiền xong rồi dừng giữa chừng.

**ACID** – bốn đảm bảo của transaction trong một database (Atomicity, Consistency, Isolation, Durability). Chữ **I** – **Isolation** nghĩa là các giao dịch chạy song song không gây xung đột hay can thiệp lẫn nhau.

**Anomaly (bất thường)** – khi thiếu isolation, hai thao tác đồng thời có thể cho kết quả sai: ví dụ hai luồng cùng đọc số dư thẻ 100k, cùng trừ 80k → số dư cuối sai (*lost update*).

**Idempotency** – một thao tác chạy lại nhiều lần vẫn cho cùng kết quả như chạy một lần. “Ghi số dư = 20k” là idempotent; “cộng thêm 20k” thì *không* (chạy 2 lần thành 40k).

Vấn đề trung tâm của chương: khi một “transaction” trải trên **nhiều service với database riêng**, ta *không còn* ACID xuyên suốt – và Saga là cách sống chung với điều đó.

## 6.1. Độ phức tạp của Giao dịch phân tán

**Khi một giao dịch trải rộng trên nhiều dịch vụ:** Trong KBM monolith ban đầu, khi sinh viên nộp bài SQL, toàn bộ luồng xử lý nằm trong một database transaction:

```
BEGIN TRANSACTION;
INSERT INTO submissions (id, user_id, sql_content) VALUES (...);
INSERT INTO judge_queue (submission_id, status) VALUES (... , 'PENDING');
UPDATE user_stats SET total_submissions = total_submissions + 1 WHERE user_id = ...;
COMMIT;
-- Tất cả thành công hoặc tất cả rollback — ACID đảm bảo
```

*Listing 6.1: Monolith transaction — ACID đảm bảo tất cả thành công hoặc rollback*

Khi KBM chuyển sang microservices, data nằm ở nhiều service/database khác nhau: Core Service quản lý submissions (database A), Judge Service quản lý execution (database B), Notification Service quản lý alerts. Không có “shared transaction” giữa chúng — database A không biết database B có commit thành công hay không.

**Tại sao 2PC không phù hợp?:**

Để hiểu lý do Saga Pattern ra đời, trước tiên chúng ta cần nhìn lại cách các hệ thống truyền thống xử lý giao dịch phân tán. Từ nhiều thập kỷ qua, **Two-Phase Commit (2PC)** là cơ chế tiêu chuẩn để giải quyết bài toán đồng bộ hóa dữ liệu trên nhiều nút mạng. Ý tưởng cốt lõi của 2PC được chia thành hai giai đoạn dưới sự dẫn dắt của một thành phần trung tâm gọi là Coordinator.

Trong giai đoạn đầu (Prepare Phase), Coordinator sẽ gửi yêu cầu tới tất cả các cơ sở dữ liệu thành phần (Participants), hỏi xem chúng đã sẵn sàng để ghi dữ liệu hay chưa. Chỉ khi tất cả các bên đồng loạt phản hồi “Sẵn sàng” và chủ động khóa (lock) dữ liệu liên quan, Coordinator mới chuyển sang giai đoạn thứ hai

(Commit Phase), ra lệnh cho tất cả cùng lúc lưu thay đổi xuống đĩa cứng. Sự phối hợp này đảm bảo đặc tính ACID quen thuộc trên môi trường phân tán.

Dưới góc nhìn lý thuyết theo Kleppmann [7, Ch.9], 2PC dường như là giải pháp an toàn để bảo toàn sự nhất quán dữ liệu ở mức độ cao nhất. Tuy nhiên, khi áp dụng cho hệ thống phân mảnh cao và yêu cầu xử lý lớn, cơ chế này bộc lộ những hạn chế lớn về hiệu năng và khả năng chịu lỗi.



Hình 6.1: Two-Phase Commit — Coordinator điều phối prepare và commit

Richardson trong [2a, Ch.4] liệt kê các lý do chính khiến 2PC (XA transactions — một chuẩn kỹ thuật phổ biến cài đặt 2PC) không phù hợp cho microservices, đặc biệt khi áp dụng vào bối cảnh hệ thống thực tế như KBM:

**Synchronous blocking** – Bộ điều phối (Coordinator) bị chặn (block) để chờ phản hồi từ tất cả các thành phần tham gia (participants); đồng thời, mỗi thành phần tham gia phải duy trì khóa giao dịch trên tài nguyên của nó cho đến khi nhận lệnh commit/abort. Trong KBM, Judge Service xử lý một bài nộp SQL có thể mất từ 5-30 giây. Điều này đồng nghĩa với việc thành phần tham gia sẽ giữ row lock và bộ điều phối bị chặn chờ trong toàn bộ khoảng thời gian đó, khiến Core Service không thể tiếp tục xử lý các yêu cầu khác.

**Single point of failure** – Nếu bộ điều phối trung tâm gặp sự cố ngừng hoạt động (crash), tất cả các thành phần tham gia sẽ bị đình trệ ở trạng thái chờ. Với KBM, nếu bộ điều phối gặp sự cố ngắt kết nối ngay trong lúc Judge đang chạy, toàn bộ tiến trình nộp bài sẽ bị đình trệ vô thời hạn.

**Reduced availability** – Giao thức yêu cầu tất cả các services tham gia phải online cùng lúc. Chẳng hạn, nếu cơ sở dữ liệu MySQL của Judge Service ngừng hoạt động, người dùng sẽ không thể nộp bài cho bất kỳ hệ quản trị cơ sở dữ liệu (DBMS) nào khác.

**NoSQL incompatible** – Nhiều công nghệ lưu trữ hiện đại không hỗ trợ XA transactions. Ví dụ, Kafka - hệ thống truyền tin cốt lõi (messaging backbone) của KBM - hoàn toàn không hỗ trợ giao thức XA, khiến việc áp dụng 2PC là bất khả thi.

**Performance bottleneck** – Việc giữ lock trong thời gian dài dẫn đến mức độ contention (cạnh tranh tài nguyên) rất cao. Trong chế độ kiểm tra (exam mode) của KBM với cường độ trên 500 lượt nộp bài mỗi phút, cơ chế khóa này sẽ tạo ra một nút thắt cổ chai (bottleneck) làm suy giảm hiệu năng toàn hệ thống. Kleppmann trong [7, Ch.9] bổ sung: 2PC là “blocking protocol” — nếu bộ điều phối gặp sự cố sau giai đoạn một, participants phải chờ vô hạn. Trong môi trường vận hành thực tế (production) với chế độ thi đấu thời gian thực, đây là một rủi ro không thể chấp nhận được.

#### ! 2PC mâu thuẫn với Microservices

Two-Phase Commit yêu cầu tất cả participants blocking và đồng bộ — điều này trực tiếp vi phạm mục tiêu triển khai độc lập (independent deployment) và tính sẵn sàng cao (high availability) của hệ thống microservices. Khi coordinator gặp sự cố, toàn bộ hệ thống bị treo cho đến khi coordinator phục hồi. Đây không phải là một hạn chế kỹ thuật có thể dùng thủ thuật để vượt qua (workaround) — đây là mâu thuẫn kiến trúc căn bản.

Sự khác biệt này đòi hỏi thay đổi tư duy thiết kế khi chuyển sang Microservices. Trong kiến trúc monolith, các cơ chế khóa đồng bộ (synchronous locking) đảm bảo tính toàn vẹn một cách tự nhiên. Khi một giao dịch bắt đầu, cơ sở dữ liệu sẽ giữ chặt các bản ghi liên quan, ngăn cản bất kỳ sự can thiệp nào từ bên ngoài cho đến khi giao dịch hoàn tất.

Tuy nhiên, trên hệ thống phân tán như KBM, nơi mạng không đáng tin cậy, việc khóa tài nguyên bằng 2PC trở thành điểm nghẽn (bottleneck). Khi quy mô tăng, xác suất tắc nghẽn cục bộ hoặc lỗi dây chuyền càng cao. Do đó, thiết kế hệ thống phải chấp nhận tính nhất quán cuối cùng (eventual consistency) thay vì nhất quán tức thời (strong consistency).

Thay vì sử dụng cơ chế khóa tài nguyên, hệ thống phân tán hiện đại ưu tiên khả năng phục hồi (resilience). Thay vì ngăn chặn lỗi hoàn toàn, hệ thống được thiết kế để phục hồi qua các giao dịch bù trừ (compensating transactions) — đó là nền tảng của Saga Pattern.

#### 💡 Compensation thay vì Locking

Nghĩ về quy trình nộp bài thực tế: sinh viên nộp SQL → hệ thống nhận bài → Judge chấm → trả kết quả. Nếu Judge gặp lỗi, chúng ta không “undo” việc nhận bài — ta *cập nhật trạng thái* thành ERROR và thông báo sinh viên thử lại. Đây chính là tư duy compensation — hành động nghiệp vụ ngược thay vì database rollback. Richardson trong [2a, Ch.4] đề cập đến quan sát của Gregor Hohpe: “Not even Starbucks uses two-phase commit” [xem Đọc thêm]. Khi khách gọi cafe, thu ngân nhận tiền rồi đưa biên lai, sau đó khách ra góc chờ Barista pha cafe (bắt đồng bộ). Nếu Starbucks dùng 2PC, thu ngân sẽ yêu cầu khách hàng chờ đợi, đồng thời phong tỏa trạng thái của cả thu ngân và nhân viên pha chế cho đến khi ly cà phê được hoàn thành rồi mới thực hiện thanh toán! Saga hoạt động y như quy trình thực tế này.

## 6.2. Saga Pattern — Chuỗi Local Transactions + Compensation

Saga hoạt động giống như quy trình xét duyệt hồ sơ qua nhiều phòng ban. Mỗi bước hoàn tất sẽ tự động kích hoạt bước tiếp theo. Nếu xảy ra lỗi giữa chừng, hệ thống sẽ chạy các nghiệp vụ bù trừ để hoàn tác lại những thay đổi trước đó.



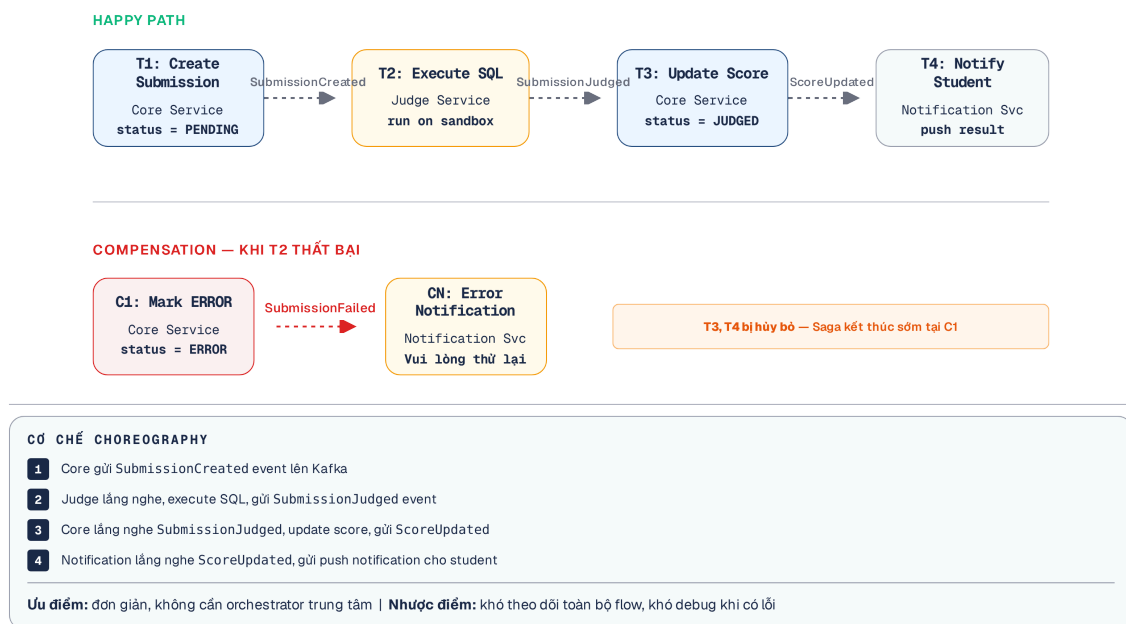
### 6.2.1. Định nghĩa

Saga là chuỗi **local transactions**, mỗi transaction cập nhật database của một service và phát đi các sự kiện hoặc thông điệp để kích hoạt giao dịch tiếp theo. Nếu một transaction thất bại, saga thực hiện **compensating transactions** để hoàn tác các thay đổi trước đó [2a, Ch.4].

### 6.2.2. KBM Submit Saga

Để làm rõ cách thức hoạt động của Saga, hãy xét luồng nghiệp vụ cốt lõi của KBM: quy trình nộp và chấm bài tập SQL. Khi sinh viên nộp bài, thay vì tạo ra một giao dịch phân tán khóa toàn bộ các bảng dữ liệu liên quan, hệ thống phân rã tiến trình này thành một chuỗi các giao dịch cục bộ độc lập (local transactions). Mỗi dịch vụ trong hệ thống sẽ tự chịu trách nhiệm cập nhật kho dữ liệu riêng biệt của dịch vụ đó, sau đó sử dụng các sự kiện (events) thông qua hệ thống message broker như Kafka để truyền tín hiệu báo hiệu cho dịch vụ tiếp theo trong chuỗi. Tuy nhiên, sự phân tách này cũng mang đến một rủi ro nhãn tiền: điều gì sẽ xảy ra nếu một bước trong chuỗi tiến trình bị gián đoạn hoặc thất bại?

Lúc này, cơ chế giao dịch bù trừ (compensating transactions) được kích hoạt. Nếu Core Service đã ghi nhận bài nộp nhưng Judge Service lại gặp lỗi khi khởi tạo sandbox, hệ thống không thể rollback database của Core Service thông qua ACID. Thay vào đó, một sự kiện bù trừ được phát đi, yêu cầu Core Service cập nhật trạng thái bài nộp thành “Lỗi” và thông báo cho sinh viên. Chuỗi các giao dịch cục bộ và giao dịch bù trừ này chính là một Saga.



Hình 6.2: KBM Submit Saga — chuỗi local transactions và compensating transactions

Richardson [2a, Ch.4] minh họa một saga điển hình qua luồng đặt hàng thương mại điện tử, trong đó bước xác thực thẻ tín dụng là một giao dịch bản lề (pivot transaction) quyết định kết quả của toàn chuỗi. Đối chiếu với KBM, luồng nộp bài (Submit Saga) có ít dịch vụ tham gia hơn (Core Service, Judge Service) nhưng độ phức tạp trong việc quản lý trạng thái vẫn tương đương.

Điểm khác biệt lớn nhất nằm ở bản chất của quá trình thực thi chấm bài (Judge execution) - một tác vụ mang tính chất **long-running** (chạy nền trong thời gian dài). Trong khi việc xác thực thẻ tín dụng chỉ mất vài mili-giây, một tiến trình chạy sandbox để biên dịch, khởi tạo dữ liệu mẫu và chạy thử các câu truy vấn SQL của sinh viên có thể kéo dài từ vài giây cho đến vài chục giây.

Sự kéo dài thời gian này vô tình mở rộng một cửa sổ bất đồng bộ (asynchronous window) đáng kể, tạo điều kiện cho các tương tác không mong muốn hoặc các lỗi mạng chen ngang. Chính thách thức đặc thù này của hệ thống giáo dục tự động đã buộc các kỹ sư phần mềm phải đặc biệt cẩn trọng khi thiết kế. Chúng ta không thể đánh đồng tất cả các bước trong chuỗi saga là như nhau, mà phải phân loại chúng một cách hệ thống dựa trên mức độ rủi ro, khả năng hoàn tác và tính tất yếu phải hoàn thành của từng bước.

### 6.2.3. Ba loại transactions trong saga

Richardson phân loại mỗi bước trong một chuỗi saga thành ba loại giao dịch riêng biệt, giúp định hình rõ ràng ranh giới và cách xử lý lỗi tại từng thời điểm. Áp dụng cơ sở lý thuyết này vào KBM Submit Saga, chúng ta có:

**Compensatable Transaction (Giao dịch có thể bù trừ)** – Đây là những bước nằm ở phần đầu của saga, có thể bị hoàn tác (undo) một cách an toàn thông qua các giao dịch bù trừ (compensating transaction). Ở ví dụ KBM, đó là T1 (Create Submission). Nếu có lỗi xảy ra ở bước sau, ta chạy C1 (Mark ERROR) để bù trừ lại.

**Pivot Transaction (Giao dịch bản lề)** – Đây là “điểm không thể quay đầu”, tương tự như thời điểm giao dịch thẻ tín dụng của khách hàng được xác thực thành công. Sau bước này, toàn bộ saga bắt buộc phải được hoàn thành và không được phép bù trừ nữa. Bản thân pivot không có bù trừ. Trong KBM, T2 (Execute SQL) chính là bước pivot: thao tác chạy mã SQL trong sandbox sẽ thành công hoặc thất bại, quyết định số phận của toàn bộ luồng.

**Retriable Transaction (Giao dịch có thể thử lại)** – Những bước đi theo sau điểm pivot. Chúng mang đặc tính là *bắt buộc* phải thành công và không cần giao dịch bù trừ. Nếu gặp lỗi kết nối hay timeout, hệ thống sẽ thực hiện thử lại (retry) liên tục cho đến khi hoàn tất. KBM có hai bước retrievable là T3 (Update Score) và T4 (Notify Student).

Việc cấu trúc đúng ba phần này là yêu cầu bắt buộc khi thiết kế Saga. Trình tự thực thi phải luôn là: bắt đầu với các bước **Compensatable**, vượt qua điểm **Pivot**, và kết thúc bằng các bước **Retriable**. Nếu đặt một bước có khả năng thất bại (nhưng không thể bù trừ) vào cuối chuỗi, hệ thống sẽ rơi vào trạng thái bất nhất. Trong luồng nộp bài của KBM, T2 (Execute SQL) là điểm Pivot. Một khi T2 thành công, saga chắc chắn phải đi đến đích. Các bước tiếp theo như T3 (Update Score) và T4 (Notify Student) là Retriable. Nếu các bước này gặp lỗi mạng hay timeout, hệ thống sẽ liên tục thử lại (infinite retries) cho đến khi thành công, và tuyệt đối không kích hoạt giao dịch bù trừ để hủy bỏ kết quả T2.

#### Hiệu đúng nghĩa 'Pivot'

T2 là pivot theo nghĩa: *nếu thành công*, saga cam kết tiến tới và không quay đầu. Cần phân biệt rõ trường hợp T2 *thất bại*: đó không phải là “compensation cho pivot” (pivot không có compensation), mà là tín hiệu để rollback các bước **compensatable** đứng trước nó — ở đây là chạy C1 để đánh dấu submission ERROR. Nói cách khác, một bước pivot thất bại sẽ kích hoạt compensation của T1, chứ bản thân pivot không bị undo.

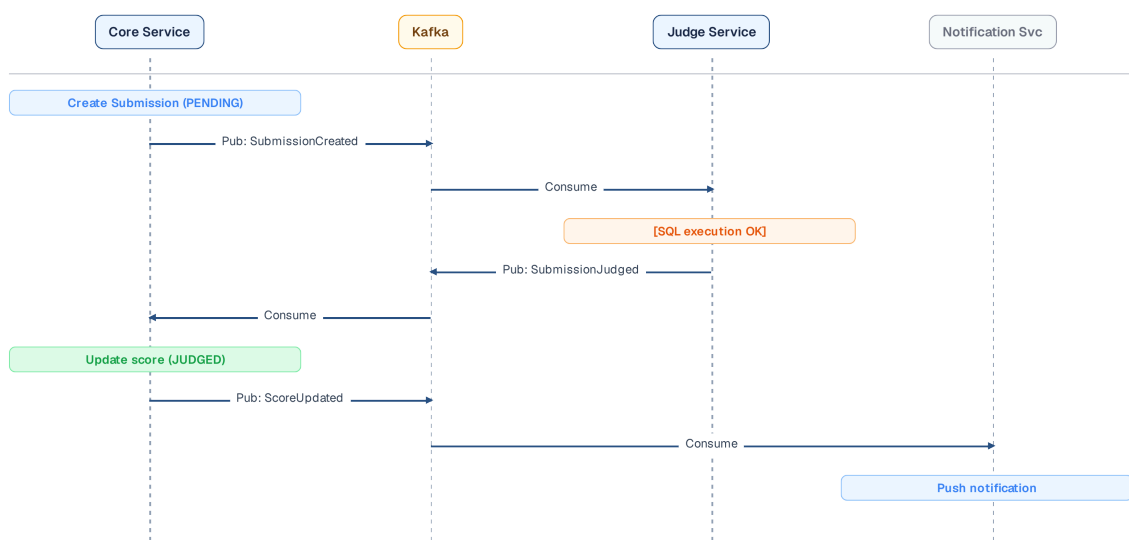
## 6.3. Choreography vs Orchestration

Để các dịch vụ phối hợp nhịp nhàng, hệ thống cần một nhạc trưởng hoặc các quy tắc tự quản. Việc chọn mô hình điều phối giống như quyết định cho sinh viên tự do đăng ký tín chỉ hay bắt buộc phải thông qua sự duyệt xét của cố vấn.

### 6.3.1. Choreography — Phi tập trung, event-driven

Khi đã định hình rõ rệt các mảnh ghép tạo nên một Saga, vấn đề tiếp theo là làm thế nào để kết nối chúng. Có hai mô hình chính để giải quyết bài toán điều phối này. Mô hình đầu tiên là **Choreography** (Vũ đạo), trong đó hệ thống hoạt động hoàn toàn phi tập trung và **không có coordinator** (bộ điều phối trung tâm). Hình ảnh so sánh gần gũi nhất là một nhóm sinh viên tổ chức sự kiện: Mỗi nhóm sinh viên trong ban tổ chức tự biết nhiệm vụ của mình; nhóm Hậu cần xong việc tự báo để nhóm Truyền thông bắt đầu, không cần một trưởng ban (coordinator) điều phối từng bước.

Đặt vào ngữ cảnh kỹ thuật, mỗi service trong hệ thống sẽ âm thầm lắng nghe các sự kiện (events) được phát ra trên một kênh truyền thông điệp chung (như Kafka), và tự động đưa ra quyết định thực thi các hành động tiếp nối dựa trên logic nội tại của chính nó [5, §4.2]. Cấu trúc lỏng lẻo, tự trị và đậm chất phản ứng (event-driven) này cũng chính là nền tảng kiến trúc gốc rễ mà hệ thống KBM hiện đang vận hành ở những phiên bản đầu tiên.



Hình 6.3: Choreography — các service tự phối hợp qua events

Mô hình Choreography rất **đơn giản** trong giai đoạn đầu vì không yêu cầu một thành phần điều phối tập trung. Các dịch vụ hoạt động hoàn toàn **loosely coupled** (ràng buộc lỏng lẻo); chúng không cần biết đến sự tồn tại của nhau mà chỉ phản ứng dựa trên các sự kiện (events).

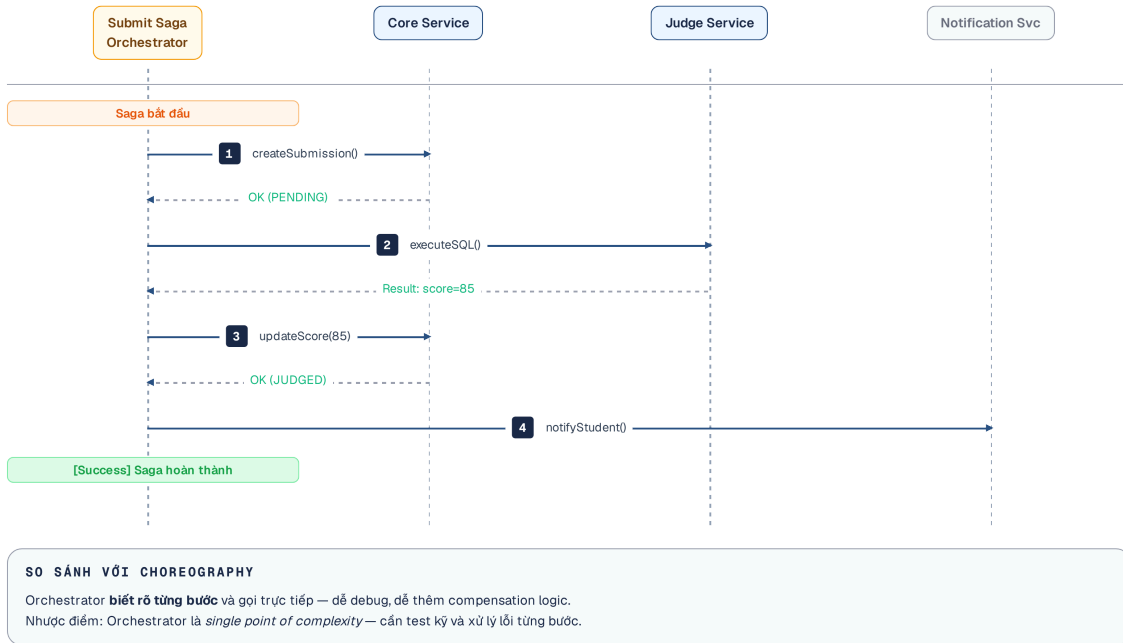
Sự độc lập này giúp quá trình **thêm mới participants** trở nên dễ dàng: một dịch vụ mới chỉ cần đăng ký (subscribe) vào luồng sự kiện tương ứng mà không làm ảnh hưởng dịch vụ cũ. Tuy nhiên, độ phức tạp sẽ tăng cao khi hệ thống mở rộng.

Luồng xử lý tổng thể dần trở nên **khó theo dõi** do logic bị phân tán. Nếu thiết kế không tốt, hệ thống dễ rơi vào **cyclic dependencies** (phụ thuộc vòng) — service A gọi B, rồi B lại gọi A. Hơn nữa, việc **kiểm thử (testing)** toàn bộ luồng đòi hỏi phải chạy đồng thời tất cả dịch vụ tham gia.

### 6.3.2. Orchestration — Tập trung, commanding

Khác với Choreography, mô hình **Orchestration** (Điều phối) sử dụng một thành phần trung tâm gọi là **Saga Orchestrator**. Orchestrator hoạt động như một Lớp trưởng hoặc Trưởng ban tổ chức, nắm quyền kiểm soát và điều phối toàn bộ luồng thực thi của Saga bằng cách trực tiếp giao việc cho từng thành viên và chờ báo cáo.

Orchestrator không chờ đợi các sự kiện xảy ra một cách thụ động, mà nó chủ động phát đi các mệnh lệnh (commands) rõ ràng đến từng dịch vụ con và cẩn thận chờ đợi phản hồi (replies) trả về để quyết định bước đi tiếp theo [2a, Ch.4]. Nếu áp dụng mô hình orchestrator tập trung vào luồng nộp bài (Submit Saga) của KBM, kiến trúc sẽ thay đổi rõ rệt.



Hình 6.4: Orchestration — saga orchestrator điều phối toàn bộ luồng xử lý

Ưu điểm lớn nhất của Orchestration là tính **đễ hiểu** và sự rõ ràng của luồng xử lý. Toàn bộ logic điều hướng được tập trung tại một nơi, giúp kỹ sư dễ dàng theo dõi trạng thái saga. Kiến trúc này loại bỏ rủi ro phụ thuộc vòng (**không có cyclic dependencies**), do biểu đồ gọi dịch vụ luôn đi theo một chiều từ orchestrator xuống các dịch vụ con.

Về mặt đảm bảo chất lượng, khâu **kiểm thử (testing)** cũng trở nên đơn giản hơn. Các lập trình viên có thể tạo ra các bản giả lập (mock) cho orchestrator để kiểm thử độc lập tính đúng đắn của từng dịch vụ con mà không cần phải dựng cả một hệ sinh thái cồng kềnh. Đối lại những lợi ích quản trị, hệ thống phải gánh vác rủi ro mang tên **centralization risk** — khi orchestrator vô tình trở thành một điểm chết tiềm năng (single point of failure).

Nếu orchestrator gặp sự cố, toàn bộ hệ thống sẽ ngừng hoạt động (single point of failure). Ngoài ra, nếu không phân chia trách nhiệm rõ ràng, hệ thống dễ rơi vào anti-pattern **smart orchestrator**: business logic bị đẩy hết vào orchestrator, biến nó thành một monolith, còn các dịch vụ con trở thành các mô hình thiếu sức sống (anemic services), chỉ đảm nhiệm các thao tác đọc/ghi cơ bản.

### 6.3.3. Khi nào dùng gì?

Rocha trong [5, §4.4] đề xuất một góc nhìn đa chiều để kết hợp cả hai cơ chế, và đây được xem là cách tiếp cận thực tế nhất:

**Saga đơn giản (2-3 bước)** — Nên ưu tiên **Choreography**. Ví dụ thực tế chính là luồng nộp bài (Submit flow) hiện tại của KBM với chỉ khoảng 3 bước ngắn gọn, việc áp dụng choreography là vừa đủ, không gây phức tạp hóa hệ thống.

**Saga phức tạp (từ 4 bước trở lên, có phân nhánh)** — **Orchestration** là giải pháp phù hợp/cần thiết. Đối với KBM, nếu trong tương lai luồng nộp bài mở rộng, cần tích hợp thêm các dịch vụ như kiểm tra

đạo văn (plagiarism check) hoặc chấm điểm dựa trên rubric, số lượng nhánh rẽ tăng lên sẽ đòi hỏi một orchestrator để kiểm soát mạch lạc.

**Giao tiếp giữa các Bounded Context** – Nên sử dụng **Choreography** để giữ sự độc lập và lỏng lẻo (loose coupling) giữa các ranh giới ngữ cảnh. Giao tiếp giữa Core Service và Judge Service trong KBM là minh chứng điển hình cho việc kết nối hai domain khác biệt.

**Nhóm phát triển chưa quen với EDA (Event-Driven Architecture)** – **Orchestration** thường dễ nắm bắt và dễ gỡ lỗi (debug) hơn, phù hợp với một đội ngũ đang trong quá trình mở rộng hoặc chưa có nhiều kinh nghiệm về luồng sự kiện phi tập trung.

Sự lựa chọn giữa hai mô hình này không bao giờ mang tính chất trắng đen rạch ròi, mà phụ thuộc hoàn toàn vào bài toán thực tiễn và giai đoạn phát triển của dự án. Theo kinh nghiệm thực chiến từ nhiều hệ thống quy mô lớn, việc duy trì một tư duy thiết kế thực dụng luôn mang lại giá trị bền vững hơn là chạy đua theo các buzzword công nghệ phức tạp.

Khi một quy trình kinh doanh có thể được giải quyết bằng vài sự kiện chuyển giao đơn giản, việc gượng ép tích hợp một bộ máy điều phối trung tâm cồng kềnh vào sẽ chỉ tiêu tốn quá mức nguồn lực của đội ngũ vận hành. Ngược lại, khi những yêu cầu mới bắt đầu xuất hiện dồn dập, làm cho biểu đồ luồng nghiệp vụ rẽ nhánh phức tạp và khó kiểm soát, việc kiên quyết duy trì cứng nhắc mô hình tự trị phi tập trung sẽ kéo cả hệ thống vào một cuộc khủng hoảng kiểm soát khó lường. Điểm mấu chốt là nhận biết đúng thời điểm “chuyển pha” của kiến trúc.

#### KBK: Choreography hay Orchestration?

Với luồng nộp bài hiện tại (3 bước, 2 dịch vụ), **choreography là đủ** — việc bổ sung thêm bộ điều phối sẽ dẫn đến tình trạng thiết kế phức tạp hóa mức cần thiết (over-engineering). Tuy nhiên, nếu tương lai luồng nghiệp vụ mở rộng (thêm plagiarism detection service, thêm grading rubric service, thêm analytics service), orchestration trở nên cần thiết. Đây là quyết định kiến trúc mở — ghi nhận để xem xét lại khi yêu cầu bài toán thay đổi.

## 6.4. Compensating Transactions & Isolation

Bù trừ dữ liệu chính là các nghiệp vụ thực tế. Khi phòng đào tạo phát hiện sai sót lịch học, họ không thể quay ngược thời gian để xóa đi, mà phải phát hành thông báo đính chính và chủ động sắp xếp lại một lịch học mới cho sinh viên.

### 6.4.1. Thiết kế compensation trong KBK

Compensation không phải là một lệnh “undo” mù quáng ở mức cơ sở dữ liệu. Về bản chất, nó là một **hành động nghiệp vụ ngược** có ý nghĩa thực tiễn [2a, Ch.4]. Việc thiết kế các hành động này phải gắn liền với logic của từng dịch vụ. Trong ngữ cảnh KBK, cơ chế này được phân rã như sau:

**T1** – Create Submission (Trạng thái PENDING): Hành động bù trừ tương ứng là C1 (Mark ERROR). Thay vì xóa bỏ (DELETE) bản ghi submission trong cơ sở dữ liệu một cách trực tiếp, hệ thống chỉ cập nhật trạng thái của nó thành ERROR. Việc này giữ lại lịch sử để hỗ trợ đối soát sau này.

**T2** – Execute SQL: Đây là điểm Pivot nên hoàn toàn không cần hành động compensation từ phía Orchestrator. Nếu tiến trình gặp lỗi, Judge Service sẽ tự động dọn dẹp (cleanup) môi trường sandbox tạm thời của nó.

**T3** – Update Score và **T4**: Send Notification: Đây là các bước Retriable, đồng nghĩa với việc chúng không bao giờ kích hoạt compensation hay rollback. Nếu gặp sự cố, hệ thống sẽ xử lý bằng cơ chế thử lại vô hạn (Infinite Retries) hoặc đẩy thông báo vào Dead Letter Queue (DLQ) để chờ can thiệp.

**Nguyên tắc cốt lõi:** Mỗi compensation phải được thiết kế sao cho nó **idempotent** (lũy đẳng) — nghĩa là có thể được gọi hàng chục lần nhưng vẫn chỉ cho ra duy nhất một kết quả như lần gọi đầu tiên. Trong thực tế của KBM, khi hàm `markError(submissionId)` bị vô tình gọi lặp lại nhiều lần do lỗi mạng, hệ thống vẫn chỉ ghi nhận trạng thái ERROR một lần duy nhất mà không sinh ra bất kỳ side effect (tác dụng phụ) nào. Lưu ý quan trọng: sau khi vượt qua Pivot transaction (T2), tất cả Retriable transactions (T3, T4) **bắt buộc** phải được thực thi đến cùng qua Infinite Retries hoặc DLQ. Chúng không được phép rollback hay kích hoạt compensate, vì lúc này Saga đã được xem là hoàn tất ở cấp độ nghiệp vụ.

#### 6.4.2. Vấn đề isolation — Anomalies

Việc sử dụng Saga cải thiện khả năng chịu lỗi và mở rộng, nhưng hệ thống phải từ bỏ tính cô lập (Isolation) của ACID. Khác với CSDL quan hệ dùng các isolation level (READ COMMITTED, SERIALIZABLE) để ngăn chặn ảnh hưởng chéo giữa các giao dịch chạy đồng thời, Saga không có cơ chế cô lập mặc định.

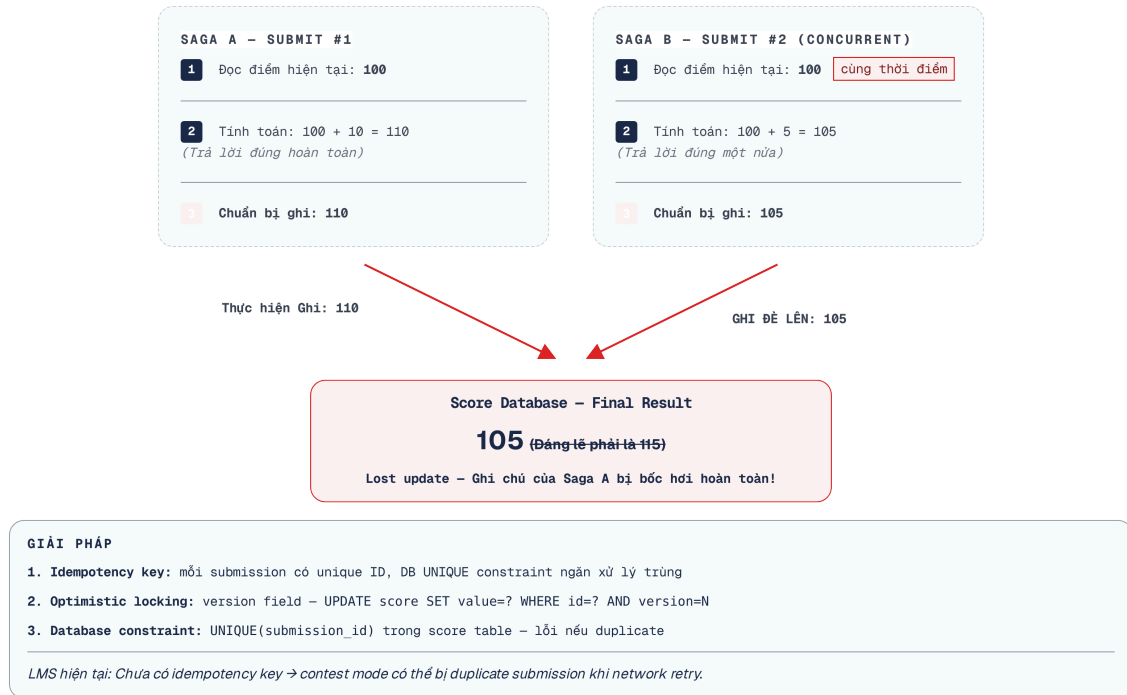
Do Saga là chuỗi các giao dịch cục bộ, kết quả trung gian chưa hoàn tất của một saga có thể bị đọc bởi các saga chạy đồng thời khác. Richardson [2a, Ch.4] gọi đây là hiện tượng **lack of isolation**. Sự thiếu vắng Isolation trực tiếp gây ra ba loại bất thường (anomalies) về dữ liệu:

##### ❗ Saga Không Có Isolation: Trade-off Cốt Lõi

Saga pattern loại bỏ blocking của 2PC nhưng hi sinh chữ “I” trong ACID — isolation. Với database truyền thống, isolation levels ngăn các transactions nhìn thấy kết quả trung gian của nhau. Trong Saga, không có cơ chế tương đương — kết quả trung gian của một saga hoàn toàn có thể bị nhìn thấy bởi các saga khác đang chạy đồng thời. Ba anomalies dưới đây là hệ quả tất yếu, cần xử lý bằng **countermeasures** thủ công.

1. **Lost Updates (Mất mát dữ liệu cập nhật)** — Hiện tượng này xảy ra khi Saga A vô tình ghi đè lên những thay đổi mà Saga B vừa mới lưu xuống thành công, trong khi Saga A hoàn toàn không có khái niệm gì về việc bản ghi đó đã bị thay đổi kể từ lúc nó đọc dữ liệu ban đầu. Đây là hệ quả tất yếu của việc các tiến trình song song cùng tranh giành quyền cập nhật một trạng thái dùng chung.





Hình 6.5: Lost Updates anomaly — hai saga ghi đè kết quả của nhau

Ví dụ sát sườn nhất trong KBM là tình huống một sinh viên kích hoạt hai lượt nộp bài liên tiếp nhau ở tốc độ cao: bài nộp A (trị giá cộng thêm 10 điểm) và bài nộp B (trị giá cộng thêm 5 điểm) vô tình được xử lý đồng thời tại cùng một thời điểm. Cả hai luồng xử lý đều tiến hành truy xuất bảng điểm hiện tại và cùng đọc được con số gốc là 100 điểm. Theo logic cá nhân, luồng A cập nhật cơ sở dữ liệu thành 110, trong khi luồng B chậm hơn một phần nghìn giây nên đã ghi đè lên thành 105. Kết quả là, thay vì sinh viên nhận được tổng số điểm xứng đáng là 115 điểm, hệ thống lại chỉ ghi nhận có 105 điểm — toàn bộ thành quả của lượt nộp A đã bị mất mát hoàn toàn không để lại dấu vết.

1. **Dirty Reads (Đọc dữ liệu rác/chưa hoàn tất)** — Kịch bản rủi ro thứ hai xảy ra khi Saga A đọc và sử dụng những dữ liệu trung gian mà Saga B vừa mới ghi xuống nhưng thực chất lại chưa hoàn thành trọn vẹn (và hoàn toàn có nguy cơ sẽ bị rollback trong tương lai gần).

Ví dụ về Dirty Read: Nếu hệ thống cộng điểm cho sinh viên ngay khi nộp bài (Saga B), Leaderboard (Saga A) lập tức đọc số điểm này để hiển thị. Tuy nhiên, nếu Judge Service sau đó gặp lỗi và kích hoạt bù trừ (trừ điểm lại), Leaderboard đã vô tình hiển thị mức điểm chưa chính thức. KBM tránh được lỗi này bằng cách đặt "Update Score" ở cuối dưới dạng giao dịch Retriable, nghĩa là điểm chỉ được cập nhật sau khi chấm xong hoàn toàn.

1. **Non-repeatable/Fuzzy Reads (Đọc dữ liệu không nhất quán)** — Tình huống thứ ba xuất hiện khi một tập dữ liệu bất ngờ thay đổi giá trị ngay giữa hai lần thực hiện truy vấn đọc trong khuôn khổ của cùng một saga duy nhất.

Trở lại với hệ thống KBM: Sinh viên mở giao diện bảng điểm và hệ thống (thực hiện truy vấn đọc lần một) báo vị trí top 3. Sinh viên tiếp tục nộp thêm bài làm tối ưu hơn. Tuy nhiên, trong đúng khoảng thời gian diễn ra quá trình submit đó, khi hệ thống thực hiện truy vấn đọc lần hai để tính toán vị trí mới, bảng điểm đã thay đổi vì các luồng saga chấm bài của các sinh viên khác đã hoàn tất. Sự bất nhất quán này là trạng thái Fuzzy Read điển hình làm suy giảm trải nghiệm làm bài.

Những rủi ro kể trên cho thấy giới hạn của các hệ thống phân tán, do đó chúng ta không nên kỳ vọng vào một giải pháp cách ly tuyệt đối.

 **ACD thay vì ACID**

Richardson trong [2a, Ch.4] chỉ ra: Sagas chỉ đảm bảo **ACD** (Atomicity, Consistency, Durability) — *không có Isolation*. Atomicity đạt được nhờ compensating transactions (rollback logic). Consistency bảo toàn bởi business rules trong mỗi local transaction. Durability do mỗi database đảm bảo. Nhưng Isolation phải được xử lý riêng — bằng **countermeasures**.

### 6.4.3. Countermeasures

Richardson đề xuất các countermeasures [2a, Ch.4], áp dụng cho KBM:

1. **Semantic Lock** — Đặt cờ “đang xử lý” trên record, ngăn saga khác đọc/ghi data chưa final:

```
// Chặn ngay từ Controller nếu user submit gấp
if(submissionRepository.existsByUserIdAndQuestionIdAndStatus(
    userId, questionId, JUDGING)) {
    throw new BadRequestException("Đang có bài chờ chấm cho câu hỏi này.");
}

// Khi saga bắt đầu — dùng Atomic SQL Update để khóa (ngăn race condition)
// UPDATE submissions SET status = 'JUDGING' WHERE id = ? AND status = 'PENDING'
int rows = submissionRepository.markAsJudging(submissionId);
if(rows == 0) {
    throw new ConcurrentModificationException("Submission đang được xử lý!");
}

// Khi saga hoàn thành — release lock (cũng dùng Atomic Update)
submissionRepository.markAsJudged(submissionId);
```

*Listing 6.2: Semantic Lock — đặt cờ “JUDGING” ngăn thao tác trên data chưa final*

1. **Commutative Updates** — Thiết kế updates không phụ thuộc thứ tự — giải quyết **lost updates**:

```
// #sym.times Không commutative: set absolute value — thứ tự quan trọng
user.setTotalScore(150);

// #sym.excl Commutative: delta update — giải quyết Lost Update
// nhưng KHÔNG lũy đẳng
// Nếu message bị gửi lại (retry), user sẽ được cộng 20, 30 điểm
user.incrementScore(+10);

// #sym.checkmark Commutative + Idempotent: Dùng DB Constraint để tránh TOCTOU
try {
    processedMessageRepository.save(new ProcessedMessage(messageId));
    user.incrementScore(+10);
} catch (DuplicateKeyException e) {
    // Đã xử lý message này, bỏ qua để đảm bảo idempotency
}
```

*Listing 6.3: Commutative Updates — delta thay vì absolute value*

Chiến thuật Commutative Updates (Cập nhật giao hoán) là một giải pháp hiệu quả cho vấn đề Lost Updates. Thay vì ghi đè một giá trị tuyệt đối (ví dụ: gán điểm số = 150), hệ thống áp dụng một phép toán thay đổi tương đối (delta, ví dụ: +10 điểm). Nhờ tính chất giao hoán của phép cộng, dù các sự kiện cập nhật điểm đến không theo thứ tự, kết quả cuối cùng vẫn chính xác. Cơ chế này rất phổ biến trong các hệ thống tính điểm, kế toán hoặc ví điện tử. Tuy nhiên, việc áp dụng cập nhật tương đối khiến hệ thống mất đi idempotency (idempotency) tự nhiên. Nếu một sự kiện (+10 điểm) bị broker phát lại hai lần, dữ liệu sẽ bị cộng dồn sai lệch.



### ⚠️ Lỗi hỏng Idempotency với Commutative Updates

Tuy phép cộng có tính giao hoán, nhưng nó KHÔNG idempotent. Vì bước Update Score là một **Retriable transaction** (được retry liên tục nếu lỗi mạng), nếu lệnh `incrementScore(+10)` bị gọi lại 3 lần, sinh viên sẽ bị cộng lặp thành 30 điểm (Double-counting)! Bắt buộc phải kết hợp với Idempotency Key (ví dụ: lưu `processed_submission_ids` trong bảng User hoặc dùng Inbox pattern) để đảm bảo không bị cộng dồn sai lệch.

Bên cạnh Lost Updates, hệ thống còn phải đối phó với hiện tượng đọc dữ liệu không nhất quán (non-repeatable reads) - nơi dữ liệu bị thay đổi bởi tiến trình khác trong lúc saga đang chạy. Để xử lý, hệ thống sử dụng:

**Pessimistic View (Góc nhìn bi quan)** — Sắp xếp lại thứ tự các bước trong saga để giảm thiểu nguy cơ dirty reads. Các bước không thể đảo ngược hoặc cập nhật tích lũy (như update score) sẽ được đặt sau pivot transaction. KBM tuân thủ nguyên tắc này khi chỉ cộng điểm sinh viên sau khi Judge ra phán quyết cuối cùng.

**Re-read Value (Đọc lại giá trị)** — Đây là hình thức áp dụng optimistic locking: thay vì dùng dữ liệu đã truy xuất từ trước, saga thực hiện đọc lại dữ liệu (re-read) ngay trước khi ghi để kiểm tra xem có thay đổi nào xảy ra không.

```
// Trước khi update score, không nên sử dụng câu lệnh 'if' để kiểm tra trên bộ nhớ ứng dụng vì dễ mắc lỗi TOCTOU
// (Time-of-Check to Time-of-Use). Phải đẩy kiểm tra xuống tầng DB:
// UPDATE scores SET value = value + 10
// WHERE user_id = ? AND NOT EXISTS (
//   SELECT 1 FROM submissions WHERE id = ? AND status = 'CANCELLED'
// )
scoreRepository.incrementIfSubmissionNotCancelled(
  userId, 10, submissionId
);
```

*Listing 6.4: Re-read Value — kiểm tra lại trước khi quyết định*

Kỹ thuật đẩy khối kiểm tra điều kiện (condition check) trực tiếp xuống lớp cơ sở dữ liệu thay vì xử lý chúng trong bộ nhớ RAM của ứng dụng là một kỹ thuật tối ưu hóa hiệu quả, giúp hệ thống tránh được lỗi Time-of-Check to Time-of-Use. Khi lệnh SQL được thực thi, cơ sở dữ liệu sẽ tự động sử dụng cơ chế khóa nội tại của nó để xác minh dữ liệu trong cùng một phân đoạn nguyên tử, đảm bảo tính toàn vẹn.

**Version File (Tập tin phiên bản)** — Để quản lý chặt chẽ những tình huống mà nhiều luồng saga tranh chấp nhau cập nhật cùng một hồ sơ, kỹ thuật cuối cùng được nhắc tới là lưu trữ lại một cuốn sổ cái ghi nhận thứ tự xuất hiện của mọi tác vụ. Từ đó cho phép hệ thống sắp xếp lại (reorder) các thao tác này theo một trật tự mong muốn trước khi áp dụng vào dữ liệu thực.

Giả sử Saga A và Saga B cùng cập nhật điểm số, Version File sẽ hoạt động như một bộ tuần tự hóa các yêu cầu. Trong KBM, một Kafka topic chuyên biệt (với cấu hình cùng một partition cho cùng một entity) chính là một Version File tự nhiên: các thông điệp điểm số đẩy vào partition này sẽ được xử lý tuần tự, loại bỏ hoàn toàn khả năng xung đột ghi đè đồng thời.

#### 6.4.4. Tổng hợp: Anomaly → Countermeasure

Để giải quyết ba anomalies trên, KBM áp dụng các rào chắn (countermeasures) tương ứng. Với tình trạng **Lost updates**, thay vì ghi đè giá trị tuyệt đối, hệ thống chuyển sang dùng **Commutative updates** (ví dụ: `incrementScore(+delta)` thay vì `setScore(value)`) để thứ tự ghi nhận điểm số không làm sai lệch kết quả cuối.

Nhằm chặn hiện tượng **Dirty reads**, KBM áp dụng **Semantic lock** bằng cách bật cờ trạng thái `JUDGING`, ngăn các tiến trình khác đọc điểm số khi chưa hoàn tất xác nhận. Cuối cùng, với **Non-repeatable reads**,

chiến thuật **Re-read value** được thực thi: ngay trước khi commit xuống database, hệ thống đọc lại một lần nữa để chắc chắn rằng quá trình xử lý bài nộp chưa bị người dùng chủ động hủy bỏ ( **CANCELLED** ).

Các kỹ thuật trên bù đắp cho sự thiếu hụt của tính cô lập (Isolation) trong hệ thống phân tán. Việc chuyển giao gánh nặng duy trì tính toàn vẹn từ cơ sở dữ liệu sang tầng ứng dụng đòi hỏi thiết kế bám sát các quy trình nghiệp vụ thực tế (business processes). Việc thiết kế các hành động bù trừ (compensation) cần phản ánh quy trình xử lý lỗi trong thực tế thay vì chỉ rollback dữ liệu cơ học.

#### ! Compensating Transaction là Hành động Nghiệp vụ

Compensation trong Saga không phải là **DELETE** hay **ROLLBACK** — đó là **hành động nghiệp vụ có ý nghĩa** phản ánh quy trình thực tế: “đánh dấu đã hủy”, “hoàn tiền”, “gửi thông báo lỗi tới user”. Điều quan trọng: mỗi compensating transaction phải được **thiết kế cùng lúc** với forward transaction, không phải là afterthought. Và tất cả compensation phải idempotent — gọi nhiều lần cho cùng kết quả.

## 6.5. Eventual Consistency — Chấp nhận và quản lý

**Định lý CAP (CAP Theorem):** Quyết định từ bỏ 2PC để chuyển sang Saga thực chất là một sự đánh đổi có chủ đích dựa trên Định lý CAP. Trong hệ thống phân tán, chúng ta không thể có cả 3 đặc tính cùng lúc: Consistency (Nhất quán), Availability (Sẵn sàng) và Partition Tolerance (Chịu lỗi mạng). Bằng cách sử dụng Saga, hệ thống của chúng ta chủ động chọn **AP** (luôn sẵn sàng tiếp nhận các yêu cầu kể cả khi kết nối mạng không ổn định) và chấp nhận hy sinh **C** (Strict Consistency - sự nhất quán tức thời).

**Từ ACID đến BASE:** Microservices chuyển từ ACID sang BASE [7, Ch.9]:

Khi chuyển đổi từ mô hình Monolith sang Microservices, hệ thống cũng bắt buộc phải dịch chuyển từ các bảo đảm chặt chẽ của ACID sang sự linh hoạt của mô hình **BASE** [7, Ch.9]. Hệ thống BASE hy sinh tính cô lập (Isolation) để đổi lấy tính sẵn sàng (Availability) và tính nhất quán cuối cùng (Eventual Consistency), trong khi Saga chính là kỹ thuật để lấy lại tính nguyên tử vĩ mô (Atomicity). Sự chuyển dịch này thể hiện rõ ràng qua từng đặc tính trong thiết kế của KBM:

**Basically Available (Sẵn sàng cơ bản)** – Hệ thống ưu tiên sự khả dụng cao: luôn trong trạng thái sẵn sàng tiếp nhận các bài nộp mới từ sinh viên, bất chấp việc một vài dịch vụ phía sau có thể đang bị gián đoạn.

**Soft state (Trạng thái mềm)** – Dữ liệu đếm số có thể tạm thời chưa được cập nhật đầy đủ và vẫn đang thay đổi trong hệ thống, hệ thống không khóa cứng các bản ghi.

**Eventual consistency (Nhất quán cuối cùng)** – Chấp nhận việc bảng xếp hạng (leaderboard) có độ trễ tạm thời, nhưng cam kết rằng *cuối cùng* mọi dữ liệu sẽ đồng bộ và chính xác.

**Durable (Bền vững)** – Cả hai hệ thống đều giữ nguyên đặc tính này. Một khi dữ liệu đã được commit, nó sẽ được lưu trữ vĩnh viễn và an toàn.

### 6.5.1. Consistency window trong KBM

Sự chuyển dịch từ ACID truyền thống sang nền tảng BASE đồng nghĩa với việc chúng ta phải làm quen và học cách sống chung với một khái niệm đầy thách thức: sự trễ nhịp của dữ liệu. Khác với cảm giác an toàn tuyệt đối khi mọi bảng dữ liệu được đồng bộ ngay tức khắc trong cùng một giao dịch nguyên khối, mô hình Microservices mở ra những khoảng thời gian quá độ, nơi mà sự chênh lệch thông tin giữa các dịch vụ là điều hiển nhiên và không thể tránh khỏi. Khoảng trễ dữ liệu này đòi hỏi các giải pháp kiến trúc để đảm bảo trải nghiệm người dùng không bị ảnh hưởng.

 Consistency Window

Rocha trong [5, Ch.5] định nghĩa **consistency window** (cửa sổ nhất quán): khoảng thời gian đếm ngược từ khoảnh khắc một sự kiện gốc rễ xảy ra cho đến khi tất cả các dịch vụ vệ tinh xung quanh phản ánh chính xác trạng thái mới nhất đó. Mục tiêu tối thượng của một kiến trúc sư không phải là triệt tiêu hoàn toàn khoảng thời gian này, mà là sử dụng kỹ thuật để giữ độ rộng của consistency window ở mức **đủ nhỏ** để người dùng cuối không nhận ra sự chậm trễ.

Trong quy trình của KBM: sau khi sinh viên nhấn nút nộp bài, consistency window mở ra và kéo dài cho tới khi có kết quả. Độ rộng của cửa sổ này phụ thuộc vào thời gian thực thi mã nguồn và tải của Judge Service. Trong khoảng thời gian chờ đó, hệ thống áp dụng các cơ chế an toàn:

- `submission.status = JUDGING` (semantic lock)
- Bảng xếp hạng (Leaderboard) tạm thời hiển thị điểm số cũ chưa được cập nhật
- Giao diện người dùng (UI) hiển thị trạng thái “Đang chấm...” nhằm cung cấp phản hồi trực quan để người dùng an tâm chờ đợi

### 6.5.2. Strategies cho UI và business

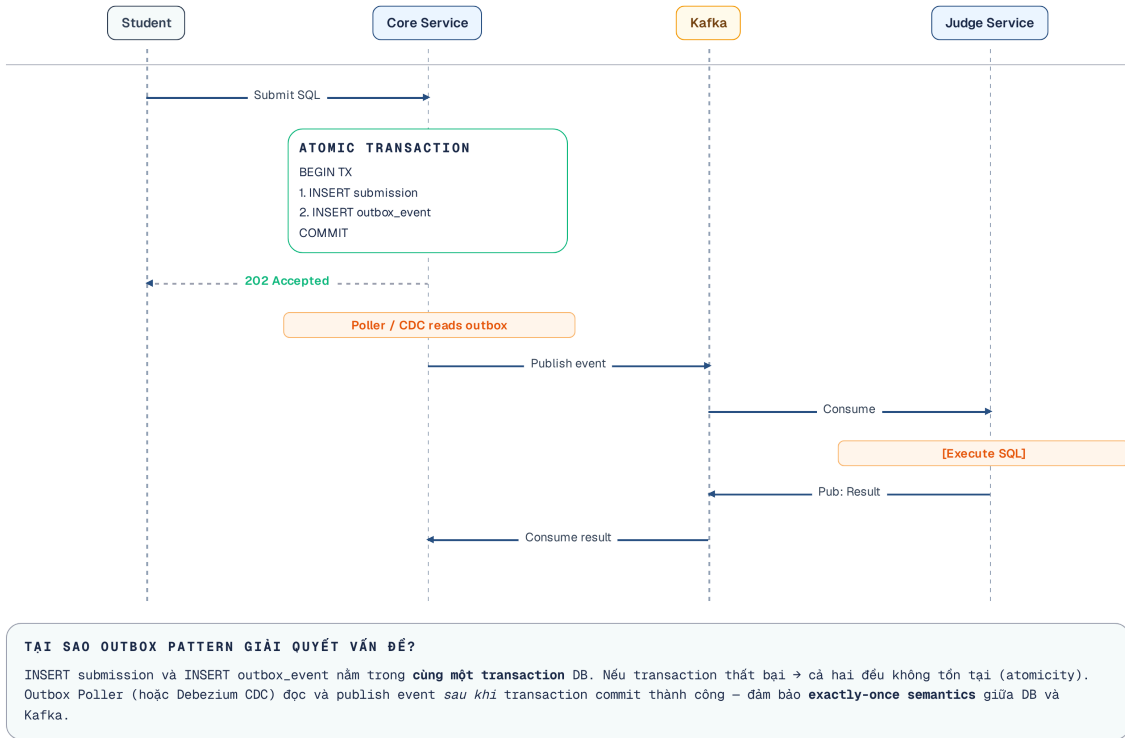
Để lấp đầy khoảng trống của “consistency window”, các kỹ sư frontend thường phải nhờ cậy đến bốn chiến thuật UI phổ biến. Bắt đầu là **Optimistic UI**: giao diện tự tin hiển thị thông báo “Bài đã gửi thành công” ngay sau khi thực hiện thao tác, xoa dịu cảm giác chờ đợi của sinh viên dù server phía sau đang thực thi chấm bài. Trong suốt quá trình đó, **Progress indicator** sẽ liên tục cập nhật các trạng thái trung gian (“Đang chấm...”, “Đang so sánh...”) để người dùng biết hệ thống không hề bị treo. Khi quá trình xử lý hoàn tất, hệ thống sử dụng cơ chế đẩy thông báo (Push notification thông qua WebSocket) để truyền kết quả cuối cùng lên màn hình. Ở kịch bản tồi tệ nhất, nếu Judge service gặp lỗi và Saga buộc phải rollback, chiến thuật **Compensation notification** sẽ được kích hoạt để hiển thị một lời xin lỗi khéo léo: “Bài không chấm được, vui lòng thử lại”, khép lại trải nghiệm người dùng một cách an toàn.

## 6.6. Case Study: KBM

Quá trình sinh viên nộp bài thực chất là một chuỗi hành động trải dài trên nhiều dịch vụ. Việc phân tích luồng xử lý này giúp chúng ta nhận diện những lỗ hổng ẩn giấu trong thiết kế giao tiếp phi tập trung của hệ thống KBM.

### 6.6.1. Hiện trạng: Implicit Saga (Choreography)

KBM hiện có một implicit saga — choreography qua Kafka events, nhưng KHÔNG được định nghĩa rõ ràng như saga:



Hình 6.6: Implicit Saga trong KBM — choreography qua Kafka events

Richardson trong [2a, Ch.4] mô tả cấu trúc Saga pattern tiêu chuẩn cần một bộ điều phối rõ ràng (explicit orchestrator), quản lý bởi state machine và tích hợp sẵn cơ chế bù trừ (compensation). Khi xem xét luồng nộp bài (Submit flow) hiện tại của KBM, thiết kế mang tính ngẫu nhiên (choreography) và thiếu các cơ chế an toàn thiết yếu (safety mechanisms). Việc thiếu saga tường minh khiến chuỗi tiến trình khó kiểm soát. Sự thiếu hụt cơ chế timeout và compensation có thể khiến một lỗi mạng cục bộ gây treo luồng nghiệp vụ trên diện rộng. Những thiếu sót này đã từng gây ra lỗi trên production, phản ánh rủi ro của hệ thống phân tán khi không xử lý tốt các tình huống ngoại lệ.

### ? Saga không Timeout tại KBM

Ban đầu, KBM sử dụng thiết kế Choreography thuần túy cho luồng chấm bài: `kbm-core` lưu trạng thái `PENDING` và đẩy event `submit.created`. Tuy nhiên, nếu Kafka mất kết nối hoặc `kbm-judge-sql` gặp sự cố ngừng hoạt động (crash), sinh viên sẽ thấy bài nộp rơi vào trạng thái chờ đợi (loading) vô thời hạn do không có dịch vụ nào phản hồi và cập nhật trạng thái.

Để khắc phục, KBM áp dụng mẫu **Timeout & Compensation**. Tuy nhiên, nếu chỉ sử dụng một tác vụ theo lịch trình ( `@Scheduled` job) để chuyển đổi trạng thái thành `TIMEOUT` một cách máy móc khi quá hạn 5 phút, hệ thống sẽ gặp lỗi **Split-Brain** (Race condition): Judge Service có thể hoàn thành việc chấm đúng 1 giây sau đó và ghi đè kết quả. Để giải quyết, Orchestrator (Core Service) **BẮT BUỘC** phải gửi một lệnh “Cancellation/Probe Command” tới Judge Service. Chỉ khi nhận được phản hồi xác nhận hủy (hoặc xác nhận chưa từng chạy) từ Judge, Core Service mới an toàn đổi trạng thái thành `TIMEOUT` và thực hiện các bước Compensation (hoàn lại lượt nộp cho sinh viên). Đây là minh chứng rõ nét: trong hệ thống phân tán, *luôn phải thiết kế cho trường hợp thành phần khác không bao giờ trả lời*.

### i Vùng xám giữa Choreography và Orchestration

Lưu ý rằng, việc Core Service chủ động gửi “Cancellation Command” (lệnh) tới Judge và lắng nghe “ACK” đã khiến nó hành xử giống như một **Lightweight Orchestrator** thay vì Choreography thuần túy (vốn chỉ phát event trung lập). Sự chuyển dịch từ Event-driven sang Command-driven này là rất phổ biến để giải quyết các luồng Timeout hoặc Compensation phức tạp.

**Phân tích các vấn đề và Đề xuất Migration:** Đối chiếu với các tiêu chuẩn thiết kế phân tán (Best Practice theo Richardson [2a, Ch.4]), luồng Submit hiện tại của KBM bộc lộ một số lỗ hổng nghiêm trọng về mặt kiến trúc. Đầu tiên là vấn đề **Implicit saga** (Saga ngầm định), nơi luồng xử lý bị phân mảnh rải rác trong mã nguồn mà thiếu đi một định nghĩa máy trạng thái (State machine) tập trung, khiến lập trình viên khó bao quát quy trình. Kế đến là sự **Thiếu cơ chế Compensation và Timeout khổng lồ**. Nếu dịch vụ Judge bất ngờ bị treo hoặc không phản hồi do nghẽn mạng, trạng thái `PENDING` của bài nộp sẽ bị đình trệ vô thời hạn. Hơn nữa, việc **Không có Semantic lock** cho phép sinh viên vô tình hoặc cố ý nộp thêm bài mới khi hệ thống vẫn đang xử lý bài cũ. Cuối cùng, sự **Thiếu theo dõi trạng thái Saga** (Saga tracking) khiến hệ thống thiếu khả năng theo dõi tiến độ hiện tại (`PENDING` → `JUDGING` → `JUDGED` / `ERROR` / `TIMEOUT`).

Để khắc phục, lộ trình nâng cấp kiến trúc được chia làm ba giai đoạn. Giai đoạn đầu tiên (**Phase 1 – Semantic Lock + Timeout**) mang tính cấp bách nhất, tập trung vào việc áp dụng khóa nghiệp vụ và cơ chế đền bù khi quá hạn nhằm tránh lỗi Split-Brain.

```

// Scheduled job kiểm tra timeout
@Scheduled(fixedRate = 60000)
public void checkSubmissionTimeouts() {
    // Tìm các bài nộp quá 5 phút chưa xong
    List<Submission> stuck = submissionRepository.findStuckSubmissions(...);
    stuck.forEach(s -> {
        // Đổi sang CANCELLING để Cronjob chu kỳ sau không gửi lệnh liên tục Cancel
        // (Chống vòng lặp vô hạn nếu dịch vụ Judge ngừng hoạt động hoàn toàn và không phản hồi ACK)
        int updated = submissionRepository.markAsCancelling(s.getId());
        if (updated > 0) {
            judgeCommandProducer.sendCancelCommand(s.getId());
        }
    });

    // Cấp 2: Hard Timeout - Nếu sau 10 phút vẫn JUDGING/CANCELLING
    // -> Ép TIMEOUT
    submissionRepository.forceTimeoutForDeadSubmissions(...);
}

// Listener nhận ACK từ Judge Service
@KafkaListener(topics = "judge.cancel.ack")
public void handleJudgeCancelAck(String submissionId) {
    // Dùng Optimistic Locking hoặc Atomic SQL thay vì `if` trên RAM
    // để chống Lost Update ngay trong code đền bù!
    int rows = submissionRepository.markAsTimeoutFromCancelling(
        submissionId
    );
    if (rows > 0) {
        Submission s = submissionRepository.findById(submissionId);
        notificationService.notify(
            s.getUserId(), "Judge timeout — thử lại tự động"
        );
    }

    // Tránh Bão Retry (Retry Storm): Không gửi object mang status
    // TIMEOUT. Clone object mới với status PENDING, và giới hạn retry.
    if (s.getRetryCount() < MAX_RETRIES) {
        Submission clone = s.cloneForRetry();
        submitProducer.sendSubmission(clone);
    } else {
        deadLetterQueue.send(s); // Quá hạn mức retry -> DLQ
    }
}
}

```

*Listing 6.5: Semantic Lock + Probe/Timeout compensation để tránh Split-Brain*

### Phase 2 — Explicit Saga Definition (đòi hỏi mức độ nỗ lực trung bình):

- Định nghĩa `SubmitSaga` class với state machine: PENDING → JUDGING → JUDGED / ERROR / TIMEOUT
- Mỗi transition kèm compensation action rõ ràng
- Log saga state changes cho auditing và debugging

### Phase 3 — Saga Orchestrator (đòi hỏi nhiều nguồn lực, khi cần mở rộng quy mô):

- Cân nhắc khi luồng nộp bài mở rộng (thêm plagiarism check, grading rubric)
- Hiện tại choreography vẫn đủ — luồng xử lý chỉ 2-3 bước
- Threshold: khi luồng nghiệp vụ > 4 bước hoặc có complex branching → chuyển sang orchestration



### 6.6.2. Workflow học vụ cũng là saga

KBM không chỉ có luồng nộp bài. Các luồng như duyệt bài tập, phê duyệt đề tài/đồ án, công bố điểm hoặc cập nhật CLO cũng có bản chất saga: nhiều bước, nhiều người tham gia, có trạng thái trung gian và có hành động bù trừ khi bị từ chối hoặc quá hạn.

Dưới lăng kính phân tích của Saga Pattern, hầu hết các nghiệp vụ cốt lõi trong trường đại học đều có thể được ánh xạ vào ba bước tiêu chuẩn: Compensatable, Pivot, và Retriable:

**Quy trình phê duyệt bài tập (Assignment approval)** – Giai đoạn đầu tiên (Compensatable) là khi giảng viên tạo bản nháp (Draft assignment) - lúc này mọi thông tin vẫn có thể hủy hoặc sửa dễ dàng. Điểm Pivot chính là khoảnh khắc giảng viên hoặc ban quản lý quyết định bấm nút phê duyệt (Approve). Một khi đã phê duyệt, hệ thống tiến vào chuỗi Retriable bắt buộc phải thành công: xuất bản bài tập ra khóa học và tự động gửi thông báo (notify) đến toàn bộ sinh viên.

**Quy trình xét duyệt đề tài/khóa luận (Thesis/project review)** – Sinh viên khởi tạo đề xuất đề tài (Submit proposal), một hành động Compensatable vì hoàn toàn có thể rút lại. Nút thắt Pivot rơi vào quyết định chính thức của hội đồng chuyên môn (Committee decision). Sau quyết định này, hệ thống sẽ thực thi các tác vụ Retriable như cập nhật trạng thái phê duyệt cuối cùng và thông báo kết quả cho cả sinh viên lẫn giảng viên hướng dẫn.

**Quy trình công bố điểm (Grade publication)** – Hệ thống bắt đầu bằng việc tính toán điểm số dự kiến (Compute provisional grade) - một bước Compensatable vì có thể bị tính lại nhiều lần. Pivot transaction xuất hiện khi giảng viên xác nhận điểm lần cuối (Lecturer confirms). Ngay sau đó, quá trình Retriable sẽ phụ trách việc công bố bảng điểm chính thức và đồng bộ dữ liệu vào mô hình đọc (read model) để phục vụ cho công tác báo cáo thống kê.

Các workflow này thường nên bắt đầu bằng **explicit state machine** hơn là orchestrator riêng. Khi số bước và nhánh tăng, orchestration mới đáng cân nhắc.

#### Interactive Demo

Xem minh họa động về **Saga Pattern (Orchestration)** tại companion web: [saga-orchestration.html](https://saga-orchestration.html)

Khi quan sát minh họa, cần lưu ý một số sai lầm rủi ro thường gặp. Việc quen với mô hình ACID từ kiến trúc nguyên khối thường khiến các kỹ sư cố gắng ép buộc tính nhất quán mạnh mẽ xuyên suốt các dịch vụ (bằng cách dùng 2PC). Hậu quả là gây ra tình trạng khóa tài nguyên chặn luồng, tạo điểm chết hệ thống và giảm tính sẵn sàng. Giải pháp là chấp nhận tính nhất quán cuối cùng ở những nơi nghiệp vụ cho phép và áp dụng Saga pattern. Bên cạnh đó, các kỹ sư thường chỉ nghĩ đến kịch bản lý tưởng (happy path) mà quên mất hành động bù trừ (compensation) cho các bước có thể hoàn tác, dẫn đến hệ thống bị kẹt ở trạng thái trung gian với dữ liệu mâu thuẫn. Mỗi bước có thể hoàn tác đều phải có hành động bù trừ tương minh và đã được kiểm thử.

Hơn nữa, nếu một Saga bắt đầu nhưng không ai kiểm tra thời gian thực thi, luồng nghiệp vụ sẽ treo vĩnh viễn khi kết nối mạng không ổn định hoặc dịch vụ ngừng hoạt động đột ngột; vì vậy, cần có tác vụ chạy ngầm kiểm tra quá hạn và tự động kích hoạt bù trừ. Việc không sử dụng khóa ngữ nghĩa (semantic lock) cho phép người dùng thao tác đúp trên dữ liệu đang xử lý (ví dụ: sinh viên nộp bài đúp khi hệ thống đang chấm) sẽ sinh ra điều kiện tranh chấp và dữ liệu rác. Các lệnh bù trừ cũng phải được thiết kế idempotent, vì nếu bị bỏ qua, hệ thống có thể bù trừ hai lần khi thực thi lại, gây sai lệch dữ liệu nhân đôi. Cuối cùng, việc lạm dụng mô hình vũ đạo (choreography) cho các Saga phức tạp nhiều nhánh sẽ tạo ra ma trận sự kiện khó theo dõi; khi quy trình vượt quá 4 bước, việc chuyển sang mô hình nhạc trưởng điều phối (orchestration) là lựa chọn sáng suốt hơn.

## 6.7. Tổng kết

Distributed transactions là bài toán khó nhất khi chuyển từ monolith sang microservices. Two-Phase Commit — giải pháp truyền thống — không phù hợp vì blocking, single point of failure, và giảm availability. Saga pattern giải quyết bằng cách thay một ACID transaction bằng chuỗi local transactions + compensating transactions. Ba loại transactions (compensatable, pivot, retrievable) tạo cấu trúc rõ ràng cho mỗi saga. Choreography (phi tập trung) phù hợp cho sagas đơn giản, orchestration (tập trung) cho sagas phức tạp.

Thiếu isolation là thách thức lớn nhất. Semantic lock, commutative updates, pessimistic view, và re-read value là bốn countermeasures giảm thiểu anomalies — tất cả đều áp dụng được cho KBM.

Eventual consistency là trade-off có chủ đích cho availability và scalability. Trong KBM, consistency window ngắn là chấp nhận được — sinh viên quen với việc chờ kết quả chấm bài. Quản lý consistency window bằng semantic lock và real-time notification là đủ cho use case hiện tại.

Phân tích KBM cho thấy hệ thống đang dùng implicit saga (choreography) và cần làm rõ compensation, semantic lock, timeout handling. Bài nộp hoặc các luồng quy trình học vụ có thể bị đình trệ ở trạng thái trung gian — đây là technical debt nghiêm trọng cần migration ưu tiên.

Ở Chương 7, chúng ta sẽ giải quyết bài toán **data management** — database-per-service, CQRS, Event Sourcing — và phân tích sâu hơn vấn đề shared database trong KBM.

### Bài tập Chương 6

Xem Phụ lục Bài tập để luyện tập:

- 6-1** – Design: Uber Ride Saga — Choreography hay Orchestration? ([L3])
- 6-2** – Trade-off: Choreography vs Orchestration ([L2])

## 6.8. Đọc thêm

### Sách tham khảo chính:

1. [2a] Chris Richardson, *Microservices Patterns*, 1st Ed. — Ch.4: Managing Transactions with Sagas
2. [5] Hugo Rocha, *Practical Event-Driven MS Architecture* — Ch.4: Sagas (Choreography, Orchestration); Ch.5: Eventual Consistency
3. [7] Martin Kleppmann, *Designing Data-Intensive Applications* — Ch.7: Transactions; Ch.9: Consistency and Consensus

### Nguồn trực tuyến:

- Gregor Hohpe, “Starbucks Does Not Use Two-Phase Commit” (2004) — [enterpriseintegrationpatterns.com](http://enterpriseintegrationpatterns.com)
- Caitie McCaffrey, “Applying the Saga Pattern” (Strange Loop 2015) — [youtube.com](https://www.youtube.com/watch?v=8vXqYkzG8j0)
- Eventuate Tram Sagas framework — [eventuate.io](https://eventuate.io)

### Tài liệu tham khảo

- Richardson, C. (2018). *Microservices Patterns: With examples in Java*, 1st Edition. Manning Publications.
- Richardson, C. (2025). *Microservices Patterns: With examples in Java*, 2nd Edition. Manning Publications.